

Text Classification with Convolutional Neural Networks (CNNs)

Introduction

- **Traditional CNNs:** CNNs excel at computer vision tasks. They detect patterns (edges, shapes, textures) in images regardless of their position.
- **Intuition for Text:** The same idea applies to text. A CNN can learn to detect meaningful patterns in sequences of words—such as key phrases, n-grams, or sentiment-bearing combinations—regardless of their exact location in a sentence.
- **Core Idea:** Instead of scanning over pixels, a 1D convolutional layer scans over a sequence of word vectors to identify

From Words to Vectors: The Input Layer

- Before a CNN can process text, words must be converted into numerical representations.
- **Word Embeddings:** Each word in the vocabulary is represented as a dense, low-dimensional vector (e.g., 50, 100, 300 dimensions).
- Pre-trained embeddings like GloVe or Word2Vec are often used. They come with semantic knowledge (e.g., "king" - "man" + "woman" $\approx$ "queen").

From Words to Vectors: The Input Layer

- Input Matrix: A sentence of n words is represented as a 2D matrix of size $n \times d$, where d is the embedding dimension.
- This matrix can be thought of as a "1D image" where the width is the number of words (n) and the "channels" are the embedding dimensions (d).

Step 1: the cat sat on the mat

- **Preprocessing the Text**
- First, we need to standardize the input. Common steps include:
- Tokenization: Split the sentence into individual words (tokens).
- ["the", "cat", "sat", "on", "the", "mat"]

Step 1: the cat sat on the mat

- Padding: CNNs require fixed-length inputs. We choose a maximum sequence length (max_len). Let's set max_len = 6. A shorter sentence would be padded with padding token, and a longer one would be truncated.
- Our processed sequence remains: ["the", "cat", "sat", "on", "the", "mat"]

Step 2: the cat sat on the mat

- **Step 2: The Embedding Lookup Layer**
- Each word is mapped to its corresponding dense vector from an embedding model. Let's assume we are using an embedding with 4 dimensions ($d=4$) for simplicity (real-world embeddings use 100-300 dimensions).

Step 2: the cat sat on the mat

- Imagine our embedding model provides the following vectors:
- the -> [0.1, 0.2, 0.3, 0.4]
- cat -> [0.5, 0.6, 0.7, 0.8]
- sat -> [0.9, 1.0, 1.1, 1.2]
- on -> [1.3, 1.4, 1.5, 1.6]
- mat -> [1.7, 1.8, 1.9, 2.0]

Step 3: the cat sat on the mat

- Step 3: Constructing the Input Matrix
- We create a matrix where each row is the vector for the corresponding word in the sequence.

Step 3: the cat sat on the mat

Word	Dimension 1	Dimension 2	Dimension 3	Dimension 4
the (index 0)	0.1	0.2	0.3	0.4
cat (index 1)	0.5	0.6	0.7	0.8
sat (index 2)	0.9	1.0	1.1	1.2
on (index 3)	1.3	1.4	1.5	1.6
the (index 4)	0.1	0.2	0.3	0.4
mat (index 5)	1.7	1.8	1.9	2.0

Step 4: the cat sat on the mat

- CNN uses a 1D convolution operation. Here's the intuition:
- **Filter (Kernel):** A small "window" of weights that slides across the rows (words) of the matrix.
- Let's use a filter of size 2 (it looks at 2 words at a time).
- This filter also has a depth of 4 (the embedding dimension), so its shape is (2, 4).

Step 4: the cat sat on the mat

- Let's say our filter W has the following weights: $\begin{bmatrix} 0.5 & 0.5 & 0.5 & 0.5 \\ -0.5 & -0.5 & -0.5 & -0.5 \end{bmatrix}$
- This filter is designed to compute a rough difference between the two word vectors it covers.

Step 4: the cat sat on the mat

- Convolution Operation:
- The filter starts at the top of the matrix and slides down one word at a time (stride=1).
- At each position, it performs an element-wise multiplication between the filter weights and the word vectors, then sums all the results into a single number. This number is a feature.

Step 4: the cat sat on the mat

- Let's calculate the first few features:
- **Position 1:** Covers ["the", "cat"]
- $(0.1*0.5 + 0.2*0.5 + 0.3*0.5 + 0.4*0.5) + (0.5*-0.5 + 0.6*-0.5 + 0.7*-0.5 + 0.8*-0.5)$
- $(0.1+0.2+0.3+0.4)*0.5 + (0.5+0.6+0.7+0.8)*-0.5$
- $(1.0*0.5) + (2.6*-0.5) = 0.5 - 1.3 = -0.8$

Step 4: the cat sat on the mat

- **Position 2:** Covers ["cat", "sat"]
 - $(0.5+0.6+0.7+0.8)*0.5 + (0.9+1.0+1.1+1.2)*-0.5$
 - $(2.6*0.5) + (4.2*-0.5) = 1.3 - 2.1 = -0.8$
- **Position 3:** Covers ["sat", "on"]
 - $(0.9+1.0+1.1+1.2)*0.5 + (1.3+1.4+1.5+1.6)*-0.5$
 - $(4.2*0.5) + (5.8*-0.5) = 2.1 - 2.9 = -0.8$
- ... and so on until the filter covers ["the", "mat"].

Step 4: the cat sat on the mat

- **Feature Map:** The result of this sliding operation is a list of features (a feature map). For our (6,4) input and a filter of size 2, we get $6 - 2 + 1 = 5$ features.
- The feature map for our example might look like: [-0.8, -0.8, -0.8, 1.0, 0.2]
- Each feature represents the presence of a specific pattern (in this case, a rough difference between adjacent words) at that specific location in the sentence.

The Nature of Text: A 1D Sequence

- **Fundamental Structure:** At its core, text is a sequence of tokens (words, subwords, or characters) occurring one after the other.
- The order of these tokens is crucial for meaning.
- "The dog bit the man" vs. "The man bit the dog" have completely different meanings despite containing the exact same words.

The Nature of Text: A 1D Sequence

- **Temporal vs. Spatial:** This sequential structure is analogous to a temporal signal, like an audio waveform or a time series.
- Each word is a "point in time."
- Convolution along this single dimension (the sequence order) is the logical way to process it.

How 1D Convolution Works on Text

- **The Input Matrix:** As shown in the previous example, a sentence is a 2D matrix of shape (sequence_length, embedding_dimension).
- **The "1D" refers to the direction of movement:** The convolution filter only slides along one axis—the sequence_length axis.
- **The Filter's Perspective:** A filter of size k (e.g., 2, 3, 5) looks at k consecutive words at a time. It performs a weighted sum

How 1D Convolution Works on Text

- **Crucial Point:** Even though the filter touches all embedding dimensions, it's still called a 1D convolution because its kernel movement and pattern detection are constrained to a single spatial/temporal dimension (the sequence of words).

1D CNNs have key advantages

- **Parameter Efficiency:** A small filter (e.g., size 3) is shared across all positions in the sequence. This drastically reduces the number of parameters compared to a dense layer that connects to every word.
- **Translation Invariance:** A CNN learns that a pattern (e.g., the n-gram "very good") is important regardless of whether it appears at the start, middle, or end of a sentence. This is a powerful inductive bias for text.

1D CNNs have key advantages

- **Hierarchical Feature Learning:** Lower layers detect simple n-grams (e.g., "not good"). Subsequent layers can combine these simple features into more complex patterns (e.g., "was definitely not good at all"), mimicking how human language is structured.

1D CNNs have key advantages

Reason	Explanation
Data Structure	Text is a 1D sequence of tokens. 1D convolution is designed for such sequential data.
Pattern Detection	It efficiently detects local patterns (n-grams) that are predictive of the output class.
Parameter Sharing	The same filter is used across all sequence positions, making the model efficient and easier to train.
Translation Invariance	It recognizes that a meaningful phrase has the same importance no matter where it appears in the text.
Dimensionality Correctness	The embedding dimensions are feature channels, not a spatial axis. 2D convolution would misuse this structure.

Code:

```
# Importing libraries
from keras.datasets import imdb
from keras.models import Sequential
from keras.layers import Dense
from keras.layers import Flatten
from keras.layers.convolutional import Conv1D
from keras.layers.convolutional import MaxPooling1D
from keras.layers.embeddings import Embedding
from keras.preprocessing import sequence
```


Code:

```
# Our dictionary will contain only of the top 7000 words appearing most frequ  
top_words = 7000
```

```
# Now we split our data-set into training and test data  
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=top_words)
```

Code:

```
# Looking at the nature of training data  
print(X_train[0])  
print(y_train[0])
```

```
print('Shape of training data: ')  
print(X_train.shape)  
print(y_train.shape)
```

```
print('Shape of test data: ')  
print(X_test.shape)  
print(y_test.shape)
```

Code:

```
# Padding the data samples to a maximum review length in words  
max_words = 450
```

```
X_train = sequence.pad_sequences(X_train, maxlen=max_words)  
X_test = sequence.pad_sequences(X_test, maxlen=max_words)
```

Code:

```
# Building the CNN Model
model = Sequential()      # initializing the Sequential nature for CNN model
```

```
# Adding the embedding layer which will take in maximum of 450 words as input
model.add(Embedding(top_words, 32, input_length=max_words))
```

Code:

```
model.add(Conv1D(32, 3, padding='same', activation='relu'))  
model.add(MaxPooling1D())  
model.add(Flatten())  
model.add(Dense(250, activation='relu'))  
model.add(Dense(1, activation='sigmoid'))
```

Code:

```
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])  
model.summary()
```

```
# Fitting the data onto model  
model.fit(X_train, y_train, validation_data=(X_test, y_test),  
          epochs=2, batch_size=128, verbose=2)
```

Code:

```
# Getting score metrics from our model  
scores = model.evaluate(X_test, y_test, verbose=0)
```

```
# Displays the accuracy of correct sentiment prediction over test data  
print("Accuracy: %.2f%%" % (scores[1]*100))
```

Implementing a 1D CNN for a Named Entity Recognition (NER) system

- The model architecture for this problem is defined as follows:
- Embedding Layer: input_dim=5 (vocab size), output_dim=3
- 1D Convolutional Layer: filters=2, kernel_size=2, activation='relu', padding='valid'
- Global Max Pooling Layer: To extract the most important feature for the entire word.
- Output Dense Layer: units=2 (number of entity tags: e.g., 0 for 'O', 1 for 'B-PER'), activation='softmax'

Implementing a 1D CNN for a Named Entity Recognition (NER) system

- A sample sentence "Artificial Intelligence" is converted into a sequence of 2 word indices based on a vocabulary:
- Character Vocabulary: {'Artificial':0, 'BERT':1, 'Intelligence':2, 'N-gram':3, ' ':4}
- Input Sequence (X): [0, 2] // representing the characters 'A' and 'I'

Pre-trained Embedding Matrix

Index	Embedding Vector	
0	[1.0, 0.0, 0.0]	// 'Artificial'
1	[0.0, 1.0, 0.0]	// 'BERT'
2	[0.0, 0.0, 1.0]	// 'Intelligence'
3	[0.5, 0.5, 0.0]	// 'N-gram'
4	[0.0, 0.0, 0.0]	// padding

Pre-trained Embedding Matrix

- A sample sentence "Artificial Intelligence" is converted into a sequence of 2 word indices based on a vocabulary:
- Character Vocabulary: {'Artificial':0, 'BERT':1, 'Intelligence':2, 'N-gram':3, ' ':4}
- Input Sequence (X): [0, 2] // representing the characters 'A' and 'I'

Pre-trained Embedding Matrix

- The weights for 1D Convolutional layer (2 filters, kernel size=2) are:
- Filter 1 (F1) - Kernel Weights (2x3 matrix):
 - $[[0.5, 0.0, 0.5], [0.0, 1.0, 0.0]]$
 - Filter 1 Bias: $b1 = 0.0$
- Filter 2 (F2) - Kernel Weights (2x3 matrix):
 - $[[0.0, 0.5, 0.0], [1.0, 0.0, 0.0]]$

Pre-trained Embedding Matrix

- The Dense layer weights are:
- Weights (W): A (2, 2) matrix: $\begin{bmatrix} 2.0 & 0.0 \\ 0.0 & 1.0 \end{bmatrix}$
- Biases (b): $[0.1, -0.1]$

Pre-trained Embedding Matrix

- Create the Input Embedding Matrix (E)
- Index 0 (Artificial): [1.0, 0.0, 0.0]
- Index 2 (Intelligence): [0.0, 0.0, 1.0]
- $E = \begin{bmatrix} [1.0, 0.0, 0.0] \\ [0.0, 0.0, 1.0] \end{bmatrix}$

Pre-trained Embedding Matrix

- Number of valid time steps
- $n_timesteps = input_sequence_length - kernel_size + 1$
- $2 - 2 + 1 = 1$
- 1 valid time step.

Calculate feature values for the first time step

- For Filter 1 (F1):
- Operation: Element-wise multiplication of the filter kernel and the input window, then sum all elements, then add bias.

Calculate feature values for the first time step

- $F1_Output = \text{sum}(\text{dot product of (Window * F1_Kernel)}) + b1$
- $= (1.0*0.5 + 0.0*0.0 + 0.0*0.5) + (0.0*0.0 + 0.0*1.0 + 1.0*0.0) + 0.0$
- $= (0.5 + 0 + 0) + (0 + 0 + 0) + 0.0$
- $= 0.5$
- **Apply ReLU:** $\text{ReLU}(0.5) = 0.5$

Calculate feature values for the first time step

- For Filter 2 (F2):
- $F2_Output = \text{sum}(\text{dot product}(\text{Window} * F2_Kernel)) + b2$
- $= (1.0*0.0 + 0.0*0.5 + 0.0*0.0) + (0.0*1.0 + 0.0*0.0 + 1.0*0.0) + 0.5$
- $= (0 + 0 + 0) + (0 + 0 + 0) + 0.5$
- $= 0.5$
- Apply ReLU: $\text{ReLU}(0.5) = 0.5$

Feature vector for the time step (before pooling)

- Feature vector for the time step (before pooling): $[0.5, 0.5]$

Global Max Pooling & Dense Layer

- Global Max Pooling Output
- Global Max Pooling takes the maximum value across each feature map (each filter's output across all time steps). We only have one time step.
- The feature map for Filter 1 is [0.5]. The max is 0.5.
- The feature map for Filter 2 is [0.5]. The max is 0.5.
- Output after Global Max Pooling: [0.5, 0.5]

Dense Layer Application

- Input features (after pooling): [0.5, 0.5]
- Dense Layer Operation: $Z = (\text{features} \bullet W) + b$
- $Z = [0.5, 0.5] \bullet \begin{bmatrix} 2.0 & 0.0 \\ 0.0 & 1.0 \end{bmatrix} + [0.1, -0.1]$
- $Z_{\text{tag0}} = (0.5 * 2.0) + (0.5 * 0.0) + 0.1 = 1.0 + 0.0 + 0.1 = 1.1$
- $Z_{\text{tag1}} = (0.5 * 0.0) + (0.5 * 1.0) + (-0.1) = 0.0 + 0.5 - 0.1 = 0.4$
- Scores (Z): [1.1, 0.4]

Apply Softmax

- $\text{softmax}(z_i) = \exp(z_i) / \sum(\exp(z_i))$
- $\exp(Z) = [\exp(1.1), \exp(0.4)] \approx [3.004, 1.492]$
- $\sum(\exp(Z)) \approx 3.004 + 1.492 = 4.496$
- $P(\text{tag0}) = 3.004 / 4.496 \approx 0.668$
- $P(\text{tag1}) = 1.492 / 4.496 \approx 0.332$
- Final Probability Distribution: $[0.668, 0.332]$